

**Name bhavya rana**

**Sapid 590034689**

**Batch 15**

## **1. Algorithm**

### **Main Program**

1. Start.
2. Declare **choice** as an integer.
3. Display the menu:
  - Increasing right-angled triangle
  - Increasing row-number triangle
  - Increasing natural-number pattern
  - Consecutive-number triangle
  - Increasing alphabet pattern
  - Exit
4. Read **choice**.
5. Use a **switch** statement:
  - If **choice** = 1, call **starTriangle()**.
  - If **choice** = 2, call **rowNumberTriangle()**.
  - If **choice** = 3, call **naturalNumberTriangle()**.
  - If **choice** = 4, call **consecutiveTriangle()**.
  - If **choice** = 5, call **alphabetTriangle()**.
  - If **choice** = 6, display the exit message.
  - Otherwise, display an invalid-choice message.
6. Repeat the menu until the user selects 6.
7. Stop.

### **Pattern 1 — Increasing Right-Angled Triangle**

1. Read the number of rows **n**.
2. If **n** <= 0, display an error message and return.
3. Set **i** = 1.
4. Repeat while **i** <= **n**:
  - Print **i** asterisks using an inner loop.
  - Move to the next line.
5. Return to the main program.

### **Pattern 2 — Increasing Row-Number Triangle**

1. Read **n**.
2. If **n** <= 0, display an error message and return.

3. For every row  $i$  from 1 to  $n$ :
  - Print the value of  $i$ ,  $i$  times.
  - Move to the next line.
4. Return.

### Pattern 3 — Increasing Natural-Number Pattern

1. Read  $n$ .
2. If  $n \leq 0$ , display an error message and return.
3. For every row  $i$  from 1 to  $n$ :
  - Print numbers from 1 through  $i$ .
  - Move to the next line.
4. Return.

### Pattern 4 — Consecutive-Number Triangle

1. Read  $n$ .
2. If  $n \leq 0$ , display an error message and return.
3. Initialize  $num = 1$ .
4. For every row  $i$  from 1 to  $n$ :
  - Repeat  $i$  times:
    - Print  $num$ .
    - Increment  $num$ .
  - Move to the next line.
5. Return.

### Pattern 5 — Increasing Alphabet Pattern

1. Read  $n$ .
2. Check whether  $1 \leq n \leq 26$ .
3. If not, display an error message and return.
4. For every row  $i$  from 1 to  $n$ :
  - Print letters starting from A up to the  $i$ th letter.
  - Move to the next line.
5. Return.

## 2. Test-Case Table

| Test Case | Menu Choice | Input $n$ | Expected Output         | Actual Output    | Result |
|-----------|-------------|-----------|-------------------------|------------------|--------|
| 1         | 1           | 4         | *, * *, * * *, * * * *  | Same as expected | Pass   |
| 2         | 2           | 4         | 1, 2 2, 3 3 3, 4 4 4 4  | Same as expected | Pass   |
| 3         | 3           | 4         | 1, 1 2, 1 2 3, 1 2 3 4  | Same as expected | Pass   |
| 4         | 4           | 4         | 1, 2 3, 4 5 6, 7 8 9 10 | Same as expected | Pass   |
| 5         | 5           | 4         | A, A B, A B C, A B C D  | Same as expected | Pass   |
| 6         | 1           | 1         | *                       | *                | Pass   |

|    |   |    |                                      |                      |      |
|----|---|----|--------------------------------------|----------------------|------|
| 7  | 1 | 0  | Error: rows must be positive         | Same error displayed | Pass |
| 8  | 5 | 26 | Alphabet pattern through Z           | Same as expected     | Pass |
| 9  | 5 | 27 | Error: rows must be between 1 and 26 | Same error displayed | Pass |
| 10 | 7 | —  | Invalid menu-choice message          | Same error displayed | Pass |
| 11 | 6 | —  | Exit program                         | Program exited       | Pass |

### 3. C Program

```
#include <stdio.h>

/* Function prototypes */
void starTriangle(void);
void rowNumberTriangle(void);
void naturalNumberTriangle(void);
void consecutiveTriangle(void);
void alphabetTriangle(void);

int main(void)
{
    int choice;

    do
    {
        printf("\n==== Increasing Patterns Menu =====\n");
        printf("1. Increasing right-angled triangle\n");
        printf("2. Increasing row-number triangle\n");
        printf("3. Increasing natural-number pattern\n");
        printf("4. Consecutive-number triangle\n");
        printf("5. Increasing alphabet pattern\n");
        printf("6. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                starTriangle();
                break;

            case 2:
```

```

        rowNumberTriangle();
        break;

    case 3:
        naturalNumberTriangle();
        break;

    case 4:
        consecutiveTriangle();
        break;

    case 5:
        alphabetTriangle();
        break;

    case 6:
        printf("Exiting program.\n");
        break;

    default:
        printf("Invalid choice. Please enter a number
from 1 to 6.\n");
    }

    } while (choice != 6);

    return 0;
}

/* 1. Increasing right-angled triangle */
void starTriangle(void)
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = 1; i <= n; i++)

```

```

    {
        for (j = 1; j <= i; j++)
        {
            printf("* ");
        }
        printf("\n");
    }
}

```

/\* 2. Increasing row-number triangle \*/

void rowNumberTriangle(void)

```

{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", i);
        }
        printf("\n");
    }
}

```

/\* 3. Increasing natural-number pattern \*/

void naturalNumberTriangle(void)

```

{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {

```

```

        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", j);
        }
        printf("\n");
    }
}

```

/\* 4. Consecutive-number triangle \*/

```

void consecutiveTriangle(void)
{
    int n, i, j;
    int num = 1;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", num);
            num++;
        }
        printf("\n");
    }
}

```

/\* 5. Increasing alphabet pattern \*/

```

void alphabetTriangle(void)
{

```

```
int n, i, j;

printf("Enter number of rows (1-26): ");
scanf("%d", &n);

if (n < 1 || n > 26)
{
    printf("Number of rows must be between 1 and 26.\n");
    return;
}

for (i = 1; i <= n; i++)
{
    for (j = 1; j <= i; j++)
    {
        printf("%c ", 'A' + j - 1);
    }
    printf("\n");
}
}
```

## 4. Compilation and Execution

```

Enter your choice: 4
Enter number of rows: 4
1
2 3
4 5 6
7 8 9 10

===== Increasing Patterns Menu =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: 5
Enter number of rows (1-26): 4
A
A B
A B C
A B C D

===== Increasing Patterns Menu =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: 7
Invalid choice. Please enter a number from 1 to 6.

===== Increasing Patterns Menu =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: █
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

```

**Result:** The program was compiled and executed successfully, and the tested outputs matched the expected increasing patterns.